

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ» (НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ, НГУ)

Факультет _____

Кафедра _____

Направление подготовки _____

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА БАКАЛАВРА

(Фамилия, Имя, Отчество автора)

Тема работы _____

«К защите допущена»

Заведующий кафедрой

ученая степень, звание

...../.....

(фамилия, И., О.) / (подпись, МП)

«.....».....20...г.

Научный руководитель

ученая степень, звание

должность, место работы

...../.....

(фамилия, И., О.) / (подпись, МП)

«.....».....20...г.

Дата защиты: «.....».....20...г.

Новосибирск, 20__

Аннотация

Работа посвящена разработке новых эвристических методов планирования запросов в реляционных СУБД. В литературе известно достаточно много алгоритмов планирования, но в промышленных приложениях известны лишь базовые подходы, опирающиеся на динамическое программирование, жадные и генетические алгоритмы и их простейшие эвристические комбинации, позволяющие выбирать тот или иной алгоритм в зависимости от числа таблиц в запросе. Данная работа преследует цель получить глубокое понимание того, как разные алгоритмы возможно комбинировать в ходе планирования запроса, делая переключение между алгоритмами в зависимости от топологии и сложности графа запроса. Ожидаемым результатом работы является решение оптимизационной задачи планирования с помощью новых эвристик, реализация решения в виде модификации планировщика в ядре реляционной СУБД с открытым кодом и экспериментальная оценка на известных промышленных бенчмарках.

Содержание

1. Введение	5
2. Формальная постановка задачи	12
3. Предшествующие работы	14
3.1. Задача выбора порядка соединений	14
3.2. Принадлежность NP	14
3.3. NP-полнота	15
3.4. Пространство поиска планов	15
3.5. Жадный подход к планированию	15
3.6. Динамическое программирование по размеру подпланов (DPsize)	17
3.7. Динамическое программирование по подмножествам (DPsub)	19
3.8. Динамическое программирование по парам связных подграфов (DPccp)	21
3.9. DPhyp: динамическое программирование на гиперграфах . . .	23
3.9.1. Гиперграф и csg-cmp-пары	24
3.9.2. Идея перечисления	24
3.9.3. DPhyp	25
3.10. Генетический подход	26
3.11. Адаптивная оптимизация больших запросов соединения . . .	27
3.12. Архитектура DP планировщика PostgreSQL	29
3.13. Оценки в модели стоимости	31
4. Ход исследования	34
4.1. Окружение	34

4.2. Подход 1. Жадные алгоритмы с различными критериями сравнения	36
4.3. Подход 2. Итеративная декомпозиция графа соединений на топологии	40
4.4. Подход 3. DPhur (реализация с открытым исходным кодом) .	43
4.5. Подход 4. Модифицированный DPhur (DPhurMod)	46
4.6. Подход 5. Связывание якорных участков	51
5. Заключение	62
5.1. Сводные результаты	62
5.2. Выводы	63
5.3. Направления дальнейшей работы	63

1. Введение

Реляционная СУБД хранит данные в виде таблиц, состоящих из строк и столбцов. Для формулирования запросов к данным используется декларативный язык SQL. С ростом объёма данных и сложности запросов увеличивается время, необходимое для их выполнения, и сокращение этого времени становится важным фактором эффективности СУБД.

Язык SQL. Стандартным средством взаимодействия с реляционными СУБД является язык SQL (Structured Query Language) — декларативный язык запросов, в котором пользователь описывает *какие* данные необходимо получить, но не указывает *как* именно их извлечь. Типичный запрос на чтение данных имеет вид `SELECT ... FROM ... WHERE ...` и задаёт: список возвращаемых атрибутов, набор таблиц-источников и условия соединения между ними, а также фильтрующие условия на строки. Один и тот же результат может быть получен множеством различных способов исполнения; выбор конкретного способа — задача оптимизатора СУБД.

В процессе трансляции запрос превращается сначала в логическое представление в виде дерева, затем с помощью оптимизатора СУБД в физический план исполнения. Производительность СУБД напрямую зависит от качества построенного физического плана. Для создания "хорошего" плана нужно решить задачу выбора оптимального порядка соединений таблиц, которая является NP-полной [1]. В процессе работы оптимизатор определяет, какой тип соединения использовать и в каком порядке соединять таблицы; запрос преобразуется в набор планов.

Соединение таблиц – это операция, которая позволяет объединять данные из двух и более таблиц по определённому условию. Использование

соединений необходимо, когда информация распределена между несколькими таблицами. Каждый аргумент это таблица, либо результат соединения нескольких таблиц.

Отношение – либо единичная таблица, либо результат соединения нескольких таблиц.

Кардинальность отношения R — это число записей в R , обозначается $|R|$. Для базовой таблицы кардинальность известна из системного каталога. Для промежуточного результата $R_i \bowtie R_j$ кардинальность неизвестна заранее и оценивается оптимизатором на основе статистик; обозначим оценку $|R_i \bowtie R_j|$.

Селективность предиката соединения p — это доля строк декартова произведения $R_i \times R_j$, удовлетворяющих условию p :

$$\text{sel}(p) = \frac{|R_i \bowtie_p R_j|}{|R_i| \cdot |R_j|}, \quad 0 \leq \text{sel}(p) \leq 1.$$

Оптимизатор оценивает $\text{sel}(p)$ по статистикам, предполагая независимость столбцов.

Виды соединений таблиц. *Внутреннее соединение* возвращает только строки, для которых условие связи выполняется в обоих аргументах. *Внешнее соединение* (левое, правое или полное) дополнительно возвращает строки одного или обоих аргументов, не имеющие совпадений, заполняя недостающие значения пустыми; в отличие от внутреннего, внешнее соединение не коммутативно и не ассоциативно уже на уровне семантики, что ограничивает множество допустимых порядков соединений.

Стадии обработки запроса. Путь от текстового SQL-запроса до результата в PostgreSQL проходит через 4 этапа:

1. **Разбор** — лексический и синтаксический анализ текста запроса, построение дерева разбора.
2. **Семантический анализ** — разрешение имён таблиц и атрибутов по системному каталогу, проверка типов, построение дерева запроса.
3. **Применение правил переписывания** — раскрытие представлений, применение правил безопасности на уровне строк; семантика запроса сохраняется.
4. **Планирование** — единая стадия, включающая предварительные логические преобразования, извлечение базовых отношений $\mathcal{R}$ и предикатов соединения $\mathcal{P}$, построение графа соединений $G = (\mathcal{R}, \mathcal{E})$ и выбор порядка соединений с назначением физических алгоритмов каждому из них. Результат — дерево операторов, передаваемое исполнителю.

Стоимостная модель. Оптимизатор СУБД использует функцию стоимости $\text{Cost} : \text{Resources}_n(T) \rightarrow \mathbb{R}_{\geq 0}$, которая сопоставляет прогнозам использования различных ресурсов (ввод-вывод, CPU, память, сетевые ресурсы) оценку ресурсозатрат плана T . Обозначим $\text{Cost}(\text{Resources}_n(T))$ как $\text{Cost}(T)$. Для этого используются статистики отношений (оценкам кардинальностей, селективностей предикатов, доступности индексов). В идеальном случае $\text{Cost}(T)$ монотонно коррелирует с фактическим временем исполнения плана T ; на практике эта корреляция нарушается из-за ошибок оценки кардинальностей промежуточных результатов [2].

Операция внутреннего соединения коммутативна и ассоциативна по результату: $(R_1 \bowtie R_2) \bowtie R_3$ и $R_1 \bowtie (R_2 \bowtie R_3)$ возвращают одно и то же множество строк. Однако *стоимость вычисления* может быть

разной: $\text{Cost}((R_1 \bowtie R_2) \bowtie R_3) \neq \text{Cost}(R_1 \bowtie (R_2 \bowtie R_3))$, поскольку кардинальности промежуточных результатов зависят от порядка соединений, и могут быть выбраны разные физические способы соединения отношений. Внешние соединения не ассоциативны и не коммутативны уже на уровне семантики: перестановка аргументов может изменить результат. Это дополнительно ограничивает множество допустимых порядков соединений.

Выбор, в какой последовательности нужно соединить таблицы, является задачей выбора порядка соединений. Различные планы исполнения для одного и того же запроса возвращают одинаковый результат, но их стоимости различны. Поэтому выбор оптимального плана позволяет сократить время отклика, минимизировать потребление ресурсов и эффективно обрабатывать большие массивы данных, значительно повышая удобство работы пользователя.

Запрос удобно представлять как граф соединений, где вершины — отношения, рёбра — условия соединений; для предикатов, затрагивающих более двух таблиц, используются гиперрёбра.

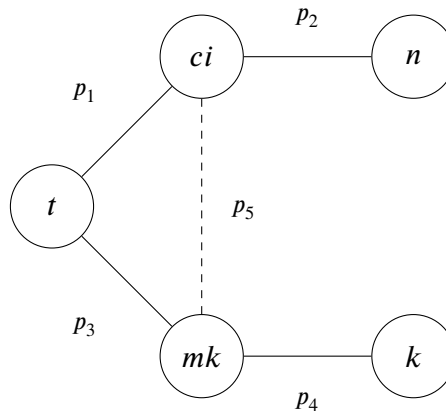
Пример запроса. Рассмотрим SQL-запрос, который находит названия фильмов жанра «комедия», вышедших после 2010 года, вместе с именами актёров:


```

SELECT t.title, n.name
FROM   title      AS t,
       cast_info  AS ci,
       name       AS n,
       movie_keyword AS mk,
       keyword    AS k
WHERE  ci.movie_id = t.id
      AND ci.person_id = n.id
      AND mk.movie_id = t.id
      AND mk.keyword_id = k.id
      AND k.keyword = 'comedy'
      AND t.production_year > 2010;

```

После стадий разбора, семантического анализа и переписывания планировщик извлекает пять базовых отношений $\mathcal{R} = \{t, ci, n, mk, k\}$ и четыре предиката соединения. Поскольку два условия содержат общий атрибут $t.id$, планировщик выводит из них класс эквивалентности $\{ci.movie_id, mk.movie_id, t.id\}$ и порождает выводимый предикат $ci.movie_id = mk.movie_id$. Граф соединений G принимает вид:



где $p_1 : ci.movie_id = t.id$, $p_2 : ci.person_id = n.id$, $p_3 : mk.movie_id = t.id$,

$p_4: mk.keyword_id = k.id$, $p_5: ci.movie_id = mk.movie_id$ (пунктирное ребро — предикат, выведенный из класса эквивалентности). Фильтрующие условия $k.keyword = 'comedy'$ и $t.production_year > 2010$ применяются локально к таблицам k и t .

На стадии физической оптимизации планировщик перебирает порядки соединений. Один из возможных планов — левостороннее дерево $T_1 = (((k \bowtie mk) \bowtie t) \bowtie ci) \bowtie n$, которое начинает с малой таблицы k (после фильтра $keyword = 'comedy'$ остаётся одна строка), соединяет с mk по индексу, затем последовательно присоединяет t , ci , n . Промежуточные результаты растут постепенно, что минимизирует объём обработки. Альтернативный порядок, в котором сначала соединяются большие таблицы ci и t , породил бы промежуточный результат в миллионы строк уже на первом шаге, многократно увеличив время исполнения.

Комбинаторная сложность. Если соединения выполнять бинарно, план задаётся полным двоичным деревом с n листьями (таблицами). Число различных форм таких деревьев — C_{n-1} , где C_k — число Каталана. С учётом перестановок таблиц по листьям число различных планов составляет $C_{n-1} \cdot n!$; асимптотически $C_{n-1} \cdot n! \sim \frac{4^{n-1} \cdot n!}{n^{3/2} \sqrt{\pi}}$. Экспоненциальный рост количества возможных планов.

Различные топологии графа (цепи, циклы, звёзды, плотные графы) приводят к разному числу допустимых планов, отличающемуся на порядки при росте числа таблиц.

На практике индустриальные СУБД комбинируют подходы. Для малого числа таблиц применяют динамическое программирование (перебор подмножеств с запоминанием лучших частичных планов). При росте количества таблиц переключаются на эвристики: жадные стратегии,

локальные улучшения, генетические алгоритмы и их простые сочетания. У такого переключения есть существенный недостаток: количество возможных планов коррелирует с числом таблиц, но линейная зависимость отсутствует.

В данной работе предложены и реализованы в ядре PostgreSQL несколько подходов, которые приводят к новому методу адаптивного планирования запросов — *Связывание якорных участков*.

2. Формальная постановка задачи

Пусть задан запрос, затрагивающий n базовых отношений $\mathcal{R} = \{R_1, \dots, R_n\}$, и множество предикатов соединения $\mathcal{P}$, определяющее граф соединений $G = (\mathcal{R}, \mathcal{E})$, где ребро $(R_i, R_j) \in \mathcal{E}$ существует тогда и только тогда, когда имеется предикат, связывающий R_i и R_j .

План исполнения запроса — это полное бинарное дерево T с n листьями, помеченными отношениями из $\mathcal{R}$, в котором каждый внутренний узел задаёт операцию соединения своих двух поддеревьев. Обозначим $\mathcal{T}_n$ — множество всех допустимых планов для данного запроса.

Алгоритм планирования. Алгоритм планирования — это отображение $\mathcal{A}: (\mathcal{R}, \mathcal{P}, \Sigma, (D)) \rightarrow \mathcal{T}_n$, где Σ — множество доступных статистик (гистограммы, список самых частых значений, оценки уникальных значений для каждого атрибута), (D) — данные. Результат $T_{\mathcal{A}} = \mathcal{A}(\mathcal{R}, \mathcal{P}, \Sigma) \in \mathcal{T}_n$ — план исполнения. Работа алгоритма $\mathcal{A}$ характеризуется двумя величинами:

- $t_{\text{plan}}(\mathcal{A})$ — время работы алгоритма планирования (время перебора, оценки и выбора плана);
- $t_{\text{exec}}(\mathcal{A})$ — время исполнения выбранного плана $T_{\mathcal{A}}$ на данных.

Время исполнения зависит от качества выбранного плана: чем ближе $T_{\mathcal{A}}$ к истинному оптимуму по фактическому времени, тем меньше t_{exec} . Время планирования определяется алгоритмом $\mathcal{A}$: точные методы (полный DP-перебор) дают малое t_{exec} , но большое t_{plan} при росте n ; быстрые эвристики сокращают t_{plan} , но могут увеличить t_{exec} .

Целевая функция. Суммарное время обработки запроса:

$$t_{\text{total}}(\mathcal{A}) = t_{\text{plan}}(\mathcal{A}) + t_{\text{exec}}(\mathcal{A}) \longrightarrow \min_{\mathcal{A}}.$$

Задача состоит в построении алгоритма планирования $\mathcal{A}^*$, минимизирующего t_{total} на заданном классе запросов $\mathcal{Q}$:

$$\mathcal{A}^* = \arg \min_{\mathcal{A}} \sum_{q \in \mathcal{Q}} t_{\text{total}}(\mathcal{A}, q).$$

3. Предшествующие работы

3.1. Задача выбора порядка соединений

В статье Ibaraki & Kameda [1] формализуется задача выбора оптимального порядка соединения n таблиц, при использовании вложенного цикла для сканирования таблиц и результатов соединений. Для заданного набора отношений и предикатов соединения выводится формула ожидаемого числа чтений страниц как функция от порядка соединяемых отношений, и требуется найти порядок, минимизирующий это значение. Авторы прямо отмечают, что задача нахождения минимального значения этой функции NP-полная.

Задача выбора оптимального вложенного порядка принимает на вход:

1. Статистики для формулы ожидаемого числа чтений страниц.
2. Описание запроса (граф соединений).
3. Пороговое значение B — существует ли такой порядок вложенности отношений, что число чтений страниц $\leq B$?

3.2. Принадлежность NP

Если мы угадали порядок отношений π , то значение функции числа чтений страниц для π вычисляется полиномиально по размеру входа. Значит, решение можно проверить за полиномиальное время, тогда задача принадлежит NP.

3.3. NP-полнота

В доказательстве NP-полноты авторы сводят к этой задаче классическую NP-полную задачу поиска полного графа [3]: по графу $G = (V, E)$ и числу k строится экземпляр запроса (в терминах отношений и условий соединения) и подбираются параметры стоимости так, что существует порядок с числом чтений страниц $\leq B$ тогда и только тогда, когда в G есть полный граф размера k .

3.4. Пространство поиска планов

Для заданного начального дерева T **пространство поиска планов** [4] — это множество всех различных порядков, которые можно получить, применяя к начальному плану последовательность из корректных преобразований (они сохраняют эквивалентность результатов, т.е семантическую корректность).

Виды корректных преобразований:

1. **Коммутативное:** $(R \bowtie_{RS} S) = (S \bowtie_{SR} R)$
2. **Ассоциативное:** $(R \bowtie_{RS} S) \bowtie_{RT,ST} T = R \bowtie_{RS,RT} (S \bowtie_{ST} T)$
3. **Левассоциативное:** $(R \bowtie_{RS} S) \bowtie_{RT,ST} T = (R \bowtie_{RT} T) \bowtie_{RS,ST} S$
4. **Правоассоциативное:** $R \bowtie_{RS,RT} (S \bowtie_{ST} T) = S \bowtie_{RS,ST} (R \bowtie_{RT} T)$

3.5. Жадный подход к планированию

В статье [5] предлагается эвристический способ построения оптимального порядка, путём ухода от экспоненциального перебора порядков соединений, строя порядок соединений жадно снизу-вверх.

Algorithm 1 $\text{GOO}(\{R_1, \dots, R_n\}, C(T_1, T_2))$

```
1: Input: set of relations to be joined
2: Input: join tree
3:  $Trees \leftarrow \{R_1, \dots, R_n\}$ 
4: while  $|Trees| \neq 1$  do
5:   find  $T_i, T_j \in Trees$  such that  $i \neq j$  and  $C(T_i, T_j)$  is minimal among all pairs
     of trees in  $Trees$ 
6:    $Trees \leftarrow Trees \setminus \{T_i\}$ 
7:    $Trees \leftarrow Trees \setminus \{T_j\}$ 
8:    $Trees \leftarrow Trees \cup \{T_i \bowtie T_j\}$ 
9: end while
10: return the (only) tree contained in  $Trees$ 
```

Преимущества:

1. Эвристика строит ветвистые деревья, что позволяет параллельно исполнять дерево плана.
2. Полиномиальность по времени планирования $O(n^3)$, где n — число начальных отношений, вместо экспоненциального времени у полного перебора.

Недостатки:

1. Нет гарантий оптимальности: локально лучший шаг может привести в глобально плохой порядок.
2. Нестабильность и зависимость от качества оценок кардинальностей/селективностей: если оценки ошибочны, жадный подход резко деградирует.

Прослеживается связь с работой [1]. Так как в общем виде задача выбора

оптимального порядка соединения отношений NP-полная, то планирование больших запросов требует больших вычислительных ресурсов, следовательно, нужны эвристики.

В современных СУБД в качестве базового подхода к планированию используется методы **динамического программирования**. Это методы точного решения задач оптимизации, которые разбивают исходную задачу на набор перекрывающихся подзадач и оптимальное решение строится из их оптимальных решений. Вместо повторного пересчёта результатов они их запоминают, снижая асимптотическую сложность по сравнению с полным перебором. Такие алгоритмы эффективны, когда пространство решений можно параметризовать состояниями (например, подмножествами, интервалами, путями в графе). Недостаток — рост памяти и экспоненциальное число состояний в худшем случае.

3.6. Динамическое программирование по размеру подпланов (DPsize)

Алгоритм DPsize[6] — вариант динамического программирования для построения оптимального ветвистого дерева соединений при условии, что граф соединений запроса связный. Алгоритм перечисляет планы в порядке возрастания размера подпланов.

DPsize поддерживает таблицу *BestPlan(S)*, сопоставляющую каждому множеству отношений *S* лучший (по минимальной стоимости) найденный план, и строит планы снизу-вверх:

Algorithm 2 DPsize($\{R_1, \dots, R_n\}, C(T_1, T_2)$)

```
1: Input: A set of relations  $R = \{R_1, \dots, R_n\}$  to be joined
2: Output: An optimal bushy join tree
3:  $B \leftarrow$  an empty DP table  $2^R \rightarrow$  join tree
4: for each  $R_i \in R$  do
5:    $B[\{R_i\}] \leftarrow R_i$ 
6: end for
7: for each  $1 < s \leq n$  ascending do
8:   for each  $S_1, S_2 \subset R$  such that  $|S_1| + |S_2| = s$  do
9:     if (not cross products  $\wedge \neg S_1$  connected to  $S_2$ )  $\vee (S_1 \cap S_2 \neq \emptyset)$  then
10:      continue
11:    end if
12:     $p_1 \leftarrow B[S_1], p_2 \leftarrow B[S_2]$ 
13:    if  $p_1 = \epsilon$  or  $p_2 = \epsilon$  then continue
14:    end if
15:     $P \leftarrow \text{CreateJoinTree}(p_1, p_2)$ 
16:    if  $B[S_1 \cup S_2] = \epsilon$  or  $C(B[S_1 \cup S_2]) > C(P)$  then
17:       $B[S_1 \cup S_2] \leftarrow P$ 
18:    end if
19:  end for
20: end for
```

Преимущества:

1. Структурированное перечисление снизу-вверх по размеру подпланов удобно для реализации.
2. Полиномиальность для простых топологий: для цепей и циклов число

внутренних проверок ($O(n^4)$).

Недостатки:

1. Экспоненциальность для сложных топологий: для звёзд и полных графов число внутренних проверок ($O(4^n)$).

3.7. Динамическое программирование по подмножествам (DPsub)

DPsub[6] описывается также как вариант динамического программирования для построения оптимального ветвистого дерева соединений для связного графа запроса. DPsub перебирает все непустые подмножества исходного множества таблиц и для каждого подмножества строит лучший план.

```

1: Input: A set of relations  $R = \{R_1, \dots, R_n\}$  to be joined
2: Output: An optimal bushy join tree
3:  $B \leftarrow$  an empty DP table  $2^R \rightarrow$  join tree
4: for each  $R_i \in R$  do
5:      $B[\{R_i\}] \leftarrow R_i$ 
6: end for
7: for each  $1 < i \leq 2^n - 1$  ascending do
8:      $S \leftarrow \{R_j \in R \mid (\lfloor i/2^{j-1} \rfloor \bmod 2) = 1\}$ 
9:     for each  $S_1, S_2 \subset S$  such that  $S_2 = S \setminus S_1$  do
10:        if (not cross products  $\wedge \neg S_1$  connected to  $S_2$ ) then
11:            continue
12:        end if
13:         $p_1 \leftarrow B[S_1], p_2 \leftarrow B[S_2]$ 
14:        if  $p_1 = \epsilon$  or  $p_2 = \epsilon$  then continue
15:        end if
16:         $P \leftarrow \text{CreateJoinTree}(p_1, p_2)$ 
17:        if  $B[S] = \epsilon$  or  $C(B[S]) > C(P)$  then
18:             $B[S] \leftarrow P$ 
19:        end if
20:    end for
21: end for

```

Преимущества:

1. Эффективней DPsize на плотных пространствах поиска(звезда/полный граф): в таких графах больше связных подмножеств и больше разбиений $S = S_1 \cup S_2$, проходящих проверки. Сложность ($O(3^n)$)

Недостатки:

1. Хуже для простых топологий (цепь/цикл): DPsub перебирает все подмножества S , но значительная часть из них несвязна. Сложность ($O(2^n)$) для цепей и ($O(n2^n)$) для циклов.

3.8. Динамическое программирование по парам связных подграфов (DPcsp)

Пусть дан граф соединений $G = (V, E)$. Введём понятие *csg-str-pair* (S_1, S_2) — в графе запроса выделим S_1, S_2 — связные непересекающиеся подграфы, такие что $S_1 \subseteq V, S_2 \subseteq V \setminus S_1$ и существует между ними ребро. (S_1, S_2) называются комплементарной парой связных подграфов.

Определим $\#csg$ — количество связных подграфов.

Определим $\#csp$ — количество комплементарных пар связанных подграфов.

В [7] предлагается алгоритм DPcsp для построения оптимального ветвистого дерева для связного графа запроса и получения нижней теоретической оценки сложности перебора, которая равна $\#csp$.

DPcsp поддерживает таблицу $BestPlan(S)$ и вместо перебора всех разбиений подмножеств рассматривает ровно комплементарные пары:

Algorithm 3 DPccp

```
1: Input: a connected query graph with relations  $R = \{R_0, \dots, R_{n-1}\}$ 
2: Output: an optimal bushy join tree
3: for all  $R_i \in R$  do
4:    $\text{BestPlan}(\{R_i\}) \leftarrow R_i$ 
5: end for
6: for all csg-cmp-pairs  $(S_1, S_2), S = S_1 \cup S_2$  do
7:    $p_1 \leftarrow \text{BestPlan}(S_1)$ 
8:    $p_2 \leftarrow \text{BestPlan}(S_2)$ 
9:    $\text{CurrPlan} \leftarrow \text{CreateJoinTree}(p_1, p_2)$ 
10:  if  $\text{cost}(\text{BestPlan}(S)) > \text{cost}(\text{CurrPlan})$  then
11:     $\text{BestPlan}(S) \leftarrow \text{CurrPlan}$ 
12:  end if
13:   $\text{CurrPlan} \leftarrow \text{CreateJoinTree}(p_2, p_1)$ 
14:  if  $\text{cost}(\text{BestPlan}(S)) > \text{cost}(\text{CurrPlan})$  then
15:     $\text{BestPlan}(S) \leftarrow \text{CurrPlan}$ 
16:  end if
17: end for
18: return  $\text{BestPlan}(\{R_0, \dots, R_{n-1}\})$ 
```

Преимущества:

1. Достижение нижней границы перебора: DPccp рассматривает ровно $\#ccp$.
2. Полиномиальность для цепей и циклов: $O(n^3)$.

Недостатки:

1. Экспоненциальность на сложных топологиях: $O(3^n)$ для полных графов

и $O(n2^n)$ для звёзд.

2. Сложность реализации: нужно эффективно перечислять *csg-сmp-pairs* без дубликатов и в порядке, корректном для DP, чтобы перед (S_1, S_2) уже были рассмотрены все непустые подмножества.

В статье алгоритм DPсср перечисляет *csg* в порядке, согласованном с DP, используя нумерацию, построенную поиском в ширину, вершин и запрет на включение вершин с меткой меньше стартовой, чтобы не порождать дубликаты. Для каждого S_1 процедура *EnumerateCmp* строит вторую часть пары S_2 только внутри дополнения $V \setminus S_1$: стартует с одиночных вершин из окрестности (S_1) и затем рекурсивно расширяет их, добавляя подмножества соседей уже построенного S_2 (то есть всегда растит связный компонент). Параметр исключения X выбирается так, чтобы S_2 не содержал вершин с меткой меньше любой вершины из S_1 (условие порядка $\min(S_1) < \min(S_2)$, где $\min(S)$ - минимальный номер вершины, входящей в S), что устраняет симметричные дубликаты $(S_1, S_2) / (S_2, S_1)$. Из-за старта из окрестности (S_1) и связного расширения гарантируется смежность S_2 к S_1 (есть ребро между компонентами), поэтому перечисляются ровно допустимые *csg-сmp-пары*.

3.9. DPhyp: динамическое программирование на гиперграфах

В другой статье, от тех же авторов [8] замечается, что DPсср эффективен для связных графов запросов, в котором все соединения внутренние и являются бинарными, т.е каждое условие включает ровно две таблицы. Однако реальные запросы содержат *сложные предикаты*, связывающие сразу несколько отношений, и внешние соединения с ограничениями на

коммутативность и ассоциативность. DPhyr обобщает DPssr на эти случаи за счёт представления запроса в виде *гиперграфа*.

3.9.1. Гиперграф и csg-стр-пары

Запрос представим гиперграфом $H = (V, E)$, где V — таблицы, а гиперребро $e = (U, W)$ связывает две непересекающиеся группы $U, W \subseteq V$, $U \cap W = \emptyset$.

Комплементарная пара определяется аналогично, только вместо рёбер гиперёбра.

3.9.2. Идея перечисления

DPhyr перечисляет аналогично только сср. Вводится линейный порядок на отношениях $<$ и правило уникальности: рассматриваются только пары с $\min(S_1) < \min(S_2)$. Порядок нужен, чтобы симметричные дубликаты не порождались.

Компоненты строятся рекурсивным расширением через *окрестность* $N(S, X)$ — разрешённые узлы, достижимые из S по гиперрёбрам, исключая запрещённые X .

Пусть есть цепочка $R_1-R_2-R_3$, цепочка $R_4-R_5-R_6$ и гиперребро между $\{R_1, R_2, R_3\}$ и $\{R_4, R_5, R_6\}$. Тогда $(S_1, S_2) = (\{R_1, R_2, R_3\}, \{R_4, R_5, R_6\})$ — допустимая csg-стр-пара: обе стороны связны и соединяемы гиперребром. Характерная ситуация: при $S_1 = \{R_1, R_2, R_3\}$ окрестность даёт вход $\min(\{R_4, R_5, R_6\}) = R_4$, и процедура EnumerateCmpRec достраивает S_2 до полной тройки, после чего пара становится соединяемой.

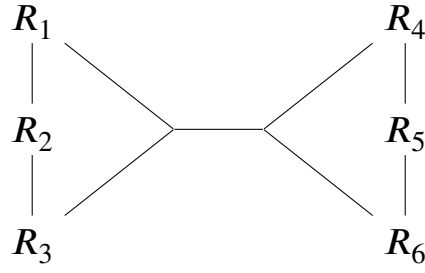


Рис. 3.1: Пример гиперграфа

3.9.3. DPhyp

Algorithm 4 DPhyp (перечисление csg-cmp-пар)

```

1: Input: Гиперграф запроса  $H = (V, E)$ , порядок  $<$  на  $V$ 
2: Output: an optimal bushy join tree.
3: global  $dp[\cdot]$   $\triangleright$  DP-таблица: множество отношений  $\rightarrow$  оптимальный план
4: function Solve
5:   for all  $v \in V$  do
6:      $dp[\{v\}] \leftarrow \text{Scan}(v)$ 
7:   end for
8:   for all  $v \in V$  in descending  $<$  do
9:      $\text{EmitCsg}(\{v\})$ 
10:     $B_v \leftarrow \{w \in V \mid w < v\} \cup \{v\}$ 
11:     $\text{EnumerateCsgRec}(\{v\}, B_v)$ 
12:   end for
13:   return  $dp[V]$ 
14: end function
15: function EnumerateCsgRec( $S_1, X$ )
16:    $Y \leftarrow \text{Neighborhood}(S_1, X)$ 
17:   for all  $N \subseteq Y$  with  $N \neq \emptyset$  do
18:     if  $\text{Connected}(S_1 \cup N)$  then
19:        $\text{EmitCsg}(S_1 \cup N)$ 
20:        $\text{EnumerateCsgRec}(S_1 \cup N, X \cup Y)$ 

```

Для внешних порядок перестановок ограничен семантикой, поэтому часть разбиений заведомо не корректна.

Идея статьи: закодировать такие ограничения в структуре гиперграфа, добавляя ограничивающие гиперребра, чтобы алгоритм DP перечислял только допустимые комбинации.

3.10. Генетический подход

В работе [9] предлагается генетический алгоритм для оптимизации запросов соединения как альтернатива классическому динамическому программированию типа DPsize. Стоимостная модель переводится в функцию приспособленности. Алгоритм пытается её минимизировать. Алгоритм поддерживает популяцию планов кандидатов, итеративно применяя селекцию, скрещивание и мутацию.

Ключевые идеи:

1. Кодирование планов:

(а) Хромосома — упорядоченный список генов, где каждый ген соответствует конкретному соединению из графа запроса, вместе с методом соединения и информацией о порядке.

2. Локальная селекция: отбор партнёров для скрещивания происходит внутри локального окружения хромосомы.

3. Операторы поиска:

(а) Мутации: случайная смена метода соединения или локальная перестановка соседних генов.

- (b) Перемещение: две перестановки и перенос случайного непрерывного фрагмента генов.

Преимущества

1. Масштабируемость: алгоритм не требует DP-таблиц экспоненциального размера, меньше потребление памяти.
2. Хорошая параллелизуемость: популяционная природа генетического подхода естественно переносится на параллельную архитектуру с небольшими коммуникационными затратами.

Недостатки

1. Нет гарантий оптимальности и стабильности: по мере роста размера запроса качество решений и устойчивость сходимости ухудшаются из-за резкого роста пространства поиска.
2. Чувствительность к параметрам: качество зависит от размера популяции, выбора операторов и схемы селекции; неудачные настройки дают деградацию.
3. Затраты на оценку приспособленности: нужно многократно вычислять стоимость планов для большого числа хромосом, что может доминировать во времени оптимизации.

3.11. Адаптивная оптимизация больших запросов соединения

В статье [10] предлагается адаптивный подход оптимизации порядка соединений, который выбирает стратегию планирования по сложности графа соединений и заданному бюджету перебора, чтобы для простых запросов

получить оптимум, а для сложных плавную и предсказуемую деградацию по времени оптимизации и качеству планов.

Ключевые идеи

1. Оценка сложности через число связных подграфов. Авторы считают $\#ср$ графа соединений. Это число совпадает с размером полной DP-таблицы. Если число связных подграфов не превышает заданного порога, то DPhur считается быстрым, и запрос оптимизируется точно.
2. Линеаризация для средних запросов. Когда точный поиск становится слишком дорогим, вводится линеаризация пространства поиска: эвристически строится линейный порядок отношений, затем DPhur ограничивается только связными подцепочками этого порядка, вместо произвольных подмножеств. Это уменьшает размер DP-таблицы с $O(2^n)$ до $O(n^2)$ и снижает сложность поиска до порядка $O(n^3)$.
3. Жадный подход + локальная оптимизация больших поддеревьев для больших запросов. Для больших запросов строится начальный план жадным методом, после чего выполняется итеративное улучшение: выбранные поддеревья перепланируются более точным методом DP, но только до размера k .

Преимущества

1. Оптимальность для простых случаев.
2. Масштабирование до тысяч отношений: при росте размера запроса происходит переход к менее дорогим стадиям.
3. Плавная деградация качества.
4. Учитывается форма графа запроса: выбор алгоритма определяется не только числом начальных отношений, что точнее отражает реальную

сложность перечисления.

Недостатки

1. Линеаризация не применима при внешних соединениях и гиперрёбрах
2. Зависимость качества от линеаризации: выбранный линейный порядок ограничивает пространство планов, некоторые хорошие порядки соединений становятся недостижимыми.
3. Локальность улучшений в GOO-DP: перепланирование поддеревьев размера k не даёт полной свободы глобальных перестановок между различными поддеревьями, поэтому итог может оставаться далёким от оптимума.
4. Пороговые параметры являются эвристическими.

3.12. Архитектура DP планировщика PostgreSQL

Практическая часть данной работы выполнена на базе СУБД PostgreSQL — распространённой реляционной системы с открытым исходным кодом.

Планировщик PostgreSQL реализует алгоритм DPsize поуровнево. Пусть дан запрос с n отношениями. Планирование выполняется в цикле по уровню $s = 2, 3, \dots, n$, где s — число базовых отношений в подплане. На каждом уровне конвейер состоит из четырёх этапов:

1. **Перечисление пар** — процедура перебирает все разбиения целевого множества S ($|S| = s$) на два непересекающихся подмножества S_1 и S_2 , для которых уже найдены подпланы, и которые связаны хотя бы одним предикатом соединения. Перебор выполняется в фиксированном порядке: сначала левосторонние планы, затем ветвистые.

2. **Построение путей** — для каждой допустимой пары (S_1, S_2) вызывается процедура, которая конструирует конкретные физические пути исполнения соединения. Эта процедура является самой дорогой частью планирования: по данным профилирования, на неё приходится свыше 90% времени t_{plan} .
3. **Инкрементальный отбор путей** — каждый построенный путь p_{new} сравнивается со всеми ранее сохранёнными путями p_{old} для того же множества S по трём независимым критериям: стоимость, порядок сортировки и параметризация. Сравнение стоимостей выполняется с допуском $\varepsilon = 1\%$: путь считается дешевле только при различии более ε по всем координатам; иначе стоимости нечётко равны. Путь p_{new} отбрасывается, если существует p_{old} , доминирующий по всем трём критериям; симметрично p_{old} удаляется, если его доминирует p_{new} . При полной эквивалентности сохраняется путь, поступивший *раньше*.
4. **Финализация уровня** — после перебора всех планов подмножеств для каждого вновь созданного множества S выбирается путь с минимальной оценочной стоимостью из числа оставшихся. Указатель на этот путь фиксируется и используется при построении подпланов на уровне $s + 1$.

Порядок перебора как скрытый параметр. Пусть для множества S рассмотрены два плана: $S = S_1^{(a)} \cup S_2^{(a)}$ (левостороннее, перебрано первым) и $S = S_1^{(b)} \cup S_2^{(b)}$ (ветвистое, перебрано позже). Обозначим построенные пути p_a и p_b соответственно. Если $\text{Cost}(p_a) \approx \text{Cost}(p_b)$ с точностью ε , то p_a остаётся в множестве сохранённых, а p_b отклоняется. При обратном порядке перебора остался бы p_b .

При точных оценках кардинальностей этот эффект незначителен,

однако при ошибочных оценках стоимости конкурирующих путей часто попадают в ε -окрестность, и порядок перебора фактически предопределяет результат. Поскольку левосторонние планы обрабатываются первыми на каждом уровне, планировщик PostgreSQL имеет систематическое структурное смещение в пользу левосторонних планов.

Число вызовов процедуры построения путей. Количество пар (S_1, S_2) , для которых вызывается процедура построения путей, определяет время планирования t_{plan} . На уровне s число допустимых разбиений зависит от топологии графа соединений G . Для полного графа из n вершин на уровне s рассматриваются $C_n^k \cdot (2^s - 2)/2$ разбиений, с максимумом при $s \approx n/2$. Распределение числа вызовов по уровням имеет характерную форму: малое число на крайних уровнях ($s = 2$ и $s = n$) и максимум на средних уровнях, где сосредоточена основная вычислительная нагрузка планирования.

3.13. Оценки в модели стоимости

В статье [2] авторы исследуют, насколько хорошо оптимизаторы промышленных СУБД справляются с выбором порядка соединений на реальных данных. Используется Join Order Benchmark (JOB) на базе данных IMDb. Модифицированный PostgreSQL получает доступ к *истинным* кардинальностям промежуточных результатов, что позволяет отделить влияние ошибок оценок от других компонентов оптимизатора.

Сбор и представление статистик. Оценки селективностей и кардинальностей строятся по статистикам. Для каждого столбца каждой таблицы вычисляются три структуры: число различных значений n_{distinct} (для оценки селективности равенства $\text{sel}(A = c)$ в предположении равномерного

распределения); список из d наиболее частых значений с их частотами; гистограмма равновысоких корзин.

Увеличение d уменьшает ошибки оценок для одиночных предикатов на одном столбце, однако *не решает* проблему оценки совместных распределений нескольких столбцов из разных таблиц. Статистики собираются независимо для каждого столбца, и оценка селективности соединения строится в предположении независимости:

$$\text{sel}(R_i.A = R_j.B) \approx \frac{1}{\max(n_{\text{distinct}}(A), n_{\text{distinct}}(B))}.$$

При наличии корреляций между атрибутами разных таблиц это предположение нарушается, и ошибки возрастают мультипликативно по цепочке соединений.

Ошибки оценок кардинальностей. Стоимостная модель оптимизатора вычисляет $\text{Cost}(T)$ по оценкам кардинальностей промежуточных результатов $|S|$ для каждого подплана S в дереве T . Авторы показывают, что для запросов с k соединениями ошибки кардинальности растут мультипликативно: если оценка каждого единичного предиката ошибается в q раз, то после k соединений накопленная ошибка может достигать q^k . На JOB для запросов с 5–17 таблицами зафиксированы ошибки в 10^2 – 10^6 раз относительно истинных кардинальностей промежуточных результатов. При подстановке истинных кардинальностей планы оптимизатора улучшаются на порядки по времени исполнения, тогда как изменение параметров самой стоимостной функции влияет значительно слабее.

Сравнение подпланов. Стоимость $\text{Cost}(T)$ задумана как величина, монотонно коррелирующая с фактическим временем исполнения t_{exec} . Однако

при ошибочных оценках кардинальностей эта корреляция нарушается: два плана T_1 и T_2 могут иметь близкие оценочные стоимости $\text{Cost}(T_1) \approx \text{Cost}(T_2)$, но кратно отличающееся фактическое время исполнения.

Leis и др. [2] показали на JOB, что при истинных кардинальностях медианная ошибка стоимостной модели PostgreSQL составляет 38%, а замена сложной модели на простую меняет среднее время исполнения на 34–41%. При этом замена оценок кардинальностей на истинные сокращает время исполнения в несколько раз. Амплитуда шума, вносимого стоимостной моделью поверх кардинальности, на сильно меньше амплитуды ошибок оценки кардинальности (10^2 – 10^6) и не способна изменить относительный порядок планов, разделённых по кардинальности.

Отсюда следует практическое правило сравнения подпланов. Для подпланов малого размера ошибки оценок кардинальностей ещё не успевают накопиться и сравнение по стоимости остаётся надёжным. Для подпланов большего размера сравнение по кардинальности оказывается устойчивее. Это правило обосновывает переключение критерия сравнения в подходе GOO_{comb} (описан в ходе исследования).

4. Ход исследования

4.1. Окружение

Все эксперименты выполнены на одной машине с фиксированной конфигурацией, чтобы обеспечить воспроизводимость и сопоставимость результатов.

Аппаратное обеспечение.

- Процессор: Intel Core Ultra 7 258V (8 ядер).
- Оперативная память: 32 ГБ.
- Операционная система: Linux Ubuntu 25.04.

СУБД. PostgreSQL версии 18.2

Бенчмарк. В данной работе в качестве Q используются два индустриальных бенчмарка: Join Order Benchmark (JOB) [2] — 113 запросов с 5–17 соединениями к базе IMDb — и JOB-Complex [11] — 30 запросов к той же базе с реалистичными свойствами: соединения по строковым атрибутам, не первичные, внешние ключи, сложные фильтры с множественными значениями. IMDb (Internet Movie Database) содержит информацию о фильмах, актёрах, ключевых словах и связанных сущностях. Запросы JOB содержат от 5 до 17 соединений, коррелированные фильтры и разнообразные топологии графов соединений, что делает бенчмарк стандартным инструментом для оценки качества планировщиков соединений.

Эти свойства приводят к катастрофическим ошибкам оценки кардинальностей: PostgreSQL не имеет совместных статистик по строковым столбцам разных таблиц, и предположение независимости нарушается на порядки. Авторы [11] показали, что PostgreSQL выбирает планы в

11,1 раза медленнее оптимальных на JOB-Complex (против 2 раз на JOB). Использование JOB-Complex позволяет оценить устойчивость предложенных подходов в условиях сильно ошибочных оценок кардинальностей.

Полученные в работе результаты относятся к конкретным $\mathcal{Q}$ (JOB и JOB-Complex) и не претендуют на существование универсального $\mathcal{A}^*$, минимизирующего t_{total} для произвольного класса запросов.

Горячие данные. Все данные были загружены в буферный кэш СУБД. Это исключает влияние дискового ввода-вывода на измерения.

Конфигурация СУБД. Параметры PostgreSQL выбраны так, чтобы устранить влияние побочных эффектов (дисковый ввод-вывод, нехватка памяти для операторов) на время исполнения запросов, изолируя эффект от выбора порядка соединений:

- `shared_buffers` = 12 GB — размер буферного кэша СУБД. Достаточен для размещения всей базы IMDb в оперативной памяти.
- `work_mem` = 8 GB — объём памяти, выделяемый каждой операции сортировки или хеширования.
- `effective_cache_size` = 8 GB — подсказка оптимизатору об объёме доступного кэша операционной системы.
- `max_parallel_workers` = 8, `max_parallel_workers_per_gather` = 8 — параметры параллелизма.

Метрики экспериментальной оценки. Прямое сравнение алгоритмов по абсолютному значению $\sum_{q \in \mathcal{Q}} t_{\text{total}}(\mathcal{A}, q)$ чувствительно к нескольким запросам с большим t_{exec} , которые могут доминировать в сумме и маскировать поведение на остальных. Поэтому в работе используются три согласованные метрики, каждая из которых отражает свой аспект близости алгоритма $\mathcal{A}$ к

решению задачи $\mathcal{A}^* = \arg \min_{\mathcal{A}} \sum_{q \in \mathcal{Q}} t_{\text{total}}(\mathcal{A}, q)$:

- **Нормализованное суммарное время** — отношение $\sum_q t_{\text{total}}(\mathcal{A}, q) / \sum_q t_{\text{total}}(\mathcal{A}_{\text{DP}}, q)$, где $\mathcal{A}_{\text{DP}}$ — стандартный DPsize PostgreSQL. Значение < 1 означает, что $\mathcal{A}$ ближе к $\mathcal{A}^*$ по целевой функции, чем базовый алгоритм.
- **Нормализованные отношения по компонентам** — те же отношения отдельно для t_{plan} и t_{exec} . Позволяют различать улучшения за счёт ускорения планирования и за счёт выбора лучшего плана.
- **Число побед** — количество запросов $q \in \mathcal{Q}$, на которых $t_{\text{total}}(\mathcal{A}, q) < t_{\text{total}}(\mathcal{A}_{\text{DP}}, q)$. Характеризует стабильность алгоритма: суммарное отношение может быть малым из-за нескольких крупных выигрышей при общей нестабильности.

4.2. Подход 1. Жадные алгоритмы с различными критериями сравнения

Мотивация. Классическая реализация использует оценочную стоимость в качестве критерия. Однако, как показано в секции «Оценки в модели стоимости», при увеличении числа соединений ошибки оценок кардинальностей каскадируются и стоимость теряет способность различать планы по истинному качеству. Возникает вопрос: можно ли подобрать критерий сравнения, более устойчивый к ошибкам оценок?

Реализованы и протестированы три варианта GOO, различающиеся только критерием выбора лучшей пары на каждом шаге.

Вариант 1: GOO по стоимости (GOO_{cost}). Критерий — оценочная

стоимость результата соединения:

$$(R_i^*, R_j^*) = \arg \min_{(R_i, R_j)} \text{Cost}(R_i \bowtie R_j).$$

Это классическая реализация GOO, в которой на каждом шаге выбирается пара с минимальной стоимостью по модели СУБД.

Вариант 2: GOO по кардинальности (GOO_{card}). Критерий — оценка объёма результата соединения:

$$(R_i^*, R_j^*) = \arg \min_{(R_i, R_j)} (|\widehat{R_i \bowtie R_j}| \cdot w(R_i \bowtie R_j)),$$

где $|\widehat{R_i \bowtie R_j}|$ — оценочная кардинальность, а $w(R_i \bowtie R_j)$ — средняя ширина строки результата (в байтах). Произведение $|\widehat{R_i \bowtie R_j}| \cdot w$ оценивает объём данных промежуточного результата и является более грубым, но более устойчивым критерием, чем стоимость на подпланах большого размера.

Вариант 3: GOO с комбинированным критерием (GOO_{comb}). Критерий адаптивно переключается в зависимости от размера аргументов соединения. Пусть $|S|$ — число базовых отношений в подплане S . Пара (R_i, R_j) называется *простой*, если $|R_i| \leq 2$ и $|R_j| \leq 2$. Критерий выбора:

$$(R_i^*, R_j^*) = \arg \min_{(R_i, R_j)} \begin{cases} \text{Cost}(R_i \bowtie R_j), & \text{если пара простая,} \\ |\widehat{R_i \bowtie R_j}| \cdot w(R_i \bowtie R_j), & \text{иначе.} \end{cases}$$

Результаты. Все три варианта протестированы на 113 запросах JOB. Суммарные отношения к DPsize:

	t_{plan}	t_{exec}	t_{total}	побед из 113
GOO_{cost}	0,85	1,44	1,22	55/113
GOO_{card}	0,86	1,62	1,34	64/113
GOO_{comb}	0,81	1,19	1,05	80/113

Все три варианта сокращают t_{plan} , но проигрывают по t_{exec} . GOO_{comb} - наибольший суммарный выигрыш.

Случай, когда жадные подходы превосходят DP. При значительных ошибках оценки кардинальностей DP может выбрать план, оптимальный по ошибочным оценкам, но неудачный по фактическому времени. Жадный алгоритм перебирает меньше вариантов, но его выбор на каждом шаге менее подвержен каскадному накоплению ошибок, поскольку опирается на локальное сравнение пар, а не на глобальную стоимость всего дерева.

Пример: запрос 6d ($t_{\text{total}}^{\text{DP}} = 2272$ мс, $t_{\text{total}}^{\text{GOO}_{\text{comb}}} = 212$ мс, улучшение в 10,7 раза). Запрос содержит фильтр по таблице n (name): name LIKE '%Downey%Robert%', оставляющий 2 строки.

DP строит левосторонний план $k \rightarrow mk \rightarrow t \rightarrow ci \rightarrow n$, в котором таблица n стоит последней. Стоимостная модель недооценивает селективность шаблонного предиката LIKE и оценивает промежуточный результат ci как небольшой. В действительности до соединения с n кардинальность достигает 785 477 строк, полученных сканированием по индексу.

GOO_{comb} на первом шаге соединяет n (2 строки после фильтра) с ci через индексное сканирование (243 строки), поскольку эта пара имеет минимальную оценку кардинальности среди всех пар. Далее результат соединяется хеш-соединением с веткой $k \rightarrow mk \rightarrow t$ (14 165 строк). Промежуточный результат после первого шага — 243 строки вместо 785 477. Время исполнения: 117 мс вместо 2163 мс.

Данный пример иллюстрирует общий механизм: когда одна из таблиц имеет высокоселективный фильтр, который стоимостная модель не в состоянии точно оценить, жадный выбор по кардинальности помещает её в начало цепочки соединений, тогда как DP откладывает её на конец.

Случаи деградации жадных подходов. Жадный алгоритм принимает на каждом шаге необратимое решение на основании локальной информации. Если выбранное соединение создаёт промежуточный результат, не позволяющий эффективно фильтровать на последующих шагах, деградация неизбежна.

Пример: запрос 17c ($t_{\text{total}}^{\text{DP}} = 3086$ мс, $t_{\text{total}}^{\text{GOO}_{\text{comb}}} = 10\,112$ мс, деградация в 3,3 раза). На ранних шагах GOO_{comb} соединяет $k \rightarrow mk \rightarrow t \rightarrow mc \rightarrow cn$: каждая пара имеет малую оценку кардинальности, и на каждом шаге выбор выглядит оптимальным. Однако построенная ветвь не содержит таблицу n (name), имеющую высокоселективный фильтр `name LIKE '%B%'`. Когда на последних шагах присоединяются ci (52,5 строки на итерацию, 148 552 итераций) и затем n (7 796 926 итераций), фильтрация по n применяется поздно, и промежуточные результаты оказываются на порядок больше, чем в плане DP.

DP находит порядок, в котором ci соединяется с n рано (ещё до mc и cn), что позволяет фильтру по имени сократить промежуточный результат до 250 строк. Жадный алгоритм не способен предвидеть этот эффект, поскольку на момент принятия решения таблица n не связана предикатом с текущим промежуточным результатом.

Результаты на JOB-Complex. На 30 запросах JOB-Complex картина меняется радикально:

	t_{plan}	t_{exec}	t_{total}	побед из 30
GOO _{cost}	0,81	1,92	1,85	9/30
GOO _{card}	0,86	0,06	0,11	17/30
GOO _{comb}	0,85	0,13	0,17	16/30

GOO_{card} показывает улучшение t_{total} в 9 раз относительно DPsize — результат, диаметрально противоположный JOB, где GOO_{card} проигрывал ($t_{\text{total}} = 1,34$). GOO_{comb} также улучшает t_{total} в 5,9 раз. Напротив, GOO_{cost} деградирует (1,85), подтверждая, что на JOB-Complex стоимостная модель сильно ошибается.

JOB-Complex содержит соединения по строковым столбцам без первичных/внешних ключей, для которых ошибки оценки кардинальностей достигают 10^5 – 10^8 . При таких ошибках DP выбирает планы со вложенными циклами, оптимальные по оценке, но катастрофические по фактическому времени. Например, запрос 12: DPsize выбирает вложенный цикл с $2,6 \cdot 10^9$ итераций ($t_{\text{exec}} = 539$ с), тогда как GOO_{card} находит хеш-соединение ($t_{\text{exec}} = 640$ мс, улучшение в 840 раз). GOO_{card}, поскольку не использует стоимостную модель и не может выбрать вложенный цикл на основании ошибочно заниженной оценки кардинальности.

4.3. Подход 2. Итеративная декомпозиция графа соединений на топологии

Мотивация. Как отмечается в [6,10], время работы алгоритмов перечисления подпланов существенно зависит от топологии графа соединений. Для цепи из n вершин DPсср перебирает $O(n^3)$ пар связных подграфов, для звезды — $O(n \cdot 2^n)$, для полного графа — $O(3^n)$. Возникает

естественная идея: если граф запроса можно разложить на подграфы с известной топологией, каждый можно спланировать специализированным алгоритмом с меньшей сложностью, чем общий DP для всего графа.

Описание подхода. Граф соединений итеративно разбивается на топологические фрагменты (плотные подграфы, звёзды, циклы, цепи) — в фиксированном порядке приоритетов. Каждый фрагмент планируется специализированным алгоритмом: цепи и циклы — интервальным DP, звёзды — планированием лучей с жадной сборкой к центру, плотные подграфы — DPsub. Результат планирования фрагмента становится вершиной нового графа; процесс повторяется до схождения к одной вершине. Финальная сборка компонент выполняется жадным алгоритмом GOO_{comb} .

Результаты. Подход протестирован на всех 113 запросах JOB и 30 запросах JOB-Complex.

Бенчмарк	t_{plan}	t_{exec}	t_{total}	побед
JOB	0,83	3,55	2,55	32/113
JOB-Complex	0,82	3,12	2,99	12/30

Экономия t_{plan} в 17% достигается за счёт меньшего числа вызовов процедуры построения путей в каждом фрагменте, но деградация t_{exec} в 3,1–3,6 раза её полностью перекрывает. Из 113 запросов JOB лишь 32 показали улучшение t_{total} ; 81 запрос деградировал, причём 10 запросов — более чем в 5 раз.

Причина деградации: потеря параметризованных путей через центральные вершины. В нормализованных реляционных схемах существуют центральные таблицы, к которым через внешние ключи присоединяются многие другие. В схеме IMDb такой вершиной является таблица t (title): к ней присоединяются ci , mc , mk , mi , mi_idx , cc . При

декомпозиции центральная попадает в один фрагмент, а соседние таблицы из других фрагментов теряют возможность использовать индексное сканирование по первичному ключу таблицы. Вместо параметризованного индексного сканирования оптимизатор вынужден использовать хеш-соединение без индекса.

Пример: запрос 10а. Запрос соединяет 7 таблиц, включая центральную вершину t и большие таблицы ci , mc (кардинальности ~ 36 млн, $\sim 2,5$ млн, $\sim 2,6$ млн). Стандартный DPsize строит левосторонний план с промежуточными кардинальностями $681 \rightarrow 4395 \rightarrow 2270 \rightarrow 76 \rightarrow 56 \rightarrow 52 \rightarrow 52$, используя индексное сканирование t по первичному ключу при каждом присоединении ($t_{\text{exec}} = 176$ мс).

Алгоритм декомпозиции относит t в один фрагмент с mc , а ci — в другой фрагмент с rt и chn . После планирования фрагментов новая вершина $\{mc, t\}$ содержит 207 410 строк (хеш-соединение двух полностью сканированных таблиц), а вершина $\{rt, ci, chn\}$ — 25 523 строки. На этапе сборки эти две новые вершины соединяются хеш-соединением по $t.id = ci.movie_id$, обрабатывая $207\,410 \times 25\,523$ кандидатов вместо 76 строк параметризованного индексного сканирования в плане DPsize. t_{exec} растёт с 176 мс до 2,66 секунд (деградация в 15,1 раза). Аналогичная потеря параметризованных путей через t наблюдается в 78 из 81 деградировавших запросов.

Вывод. Подход с декомпозицией на топологии бесперспективен для схем с выраженными центральными вершинами, характерными для нормализованных реляционных баз данных. Корень проблемы — разрушение параметризованных индексных путей при изоляции центральных вершин в один фрагмент. Это наблюдение мотивирует *безопасную* стратегию сокращения пространства поиска, в которой центральные не могут быть

изолированы от своих соседей.

4.4. Подход 3. DPhur (реализация с открытым исходным кодом)

Мотивация. Алгоритм DPhur [8] достигает нижней теоретической границы перебора среди DP-алгоритмов для произвольных графов соединений: он перечисляет *только* пары связных подграфов (csg-сmp-pairs), тогда как DPsize перебирает все подмножества, проверяя связность апостериори. Для разреженных графов (цепи, звёзды) это может дать экспоненциальное сокращение числа рассматриваемых пар.

Помимо теоретической границы перебора, DPhur меняет *порядок*, в котором кандидаты поступают в процедуру построения путей: DPsize обрабатывает подпланы по возрастанию размера, а DPhur — обходом по соседям гиперграфа. Поскольку результат планирования в PostgreSQL зависит от порядка перебора (см. «Архитектура планировщика PostgreSQL»), эксперимент позволяет одновременно проверить две гипотезы: (1) приводит ли достижение нижней теоретической границы перебора к улучшению t_{plan} и (2) может ли альтернативный порядок обхода систематически улучшить t_{exec} .

Описание. Использована реализация DPhur с открытым исходным кодом, интегрированная в PostgreSQL в виде расширения. Алгоритм строит гиперграф соединений и перечисляет пары связных подграфов (S_1, S_2) обходом по соседям, гарантируя, что каждая перечисленная пара связна и соединена хотя бы одним предикатом. Для каждой пары вызывается процедура построения путей, аналогичная стандартному DPsize.

Результаты. Суммарные отношения к стандартному DPsize:

- t_{plan} : 1,015 (деградация 1,5%),
- t_{exec} : 1,131 (деградация 13,1%),
- t_{total} : 1,088 (деградация 8,8%).

Алгоритм, достигающий нижней теоретической границы перебора, показал результат *хуже* стандартного DPsize по всем трём метрикам.

Результаты на JOB-Complex. На 30 запросах JOB-Complex DPhur также уступает DPsize: $t_{\text{plan}} = 1,06$, $t_{\text{exec}} = 1,10$, $t_{\text{total}} = 1,10$, побед 8 из 30. При этом на отдельных запросах с катастрофически плохими планами DPsize (запросы 5, 11, 12, 20, где DPsize выбирает планы с вложенными циклами с 10^9 итераций) DPhur уступает им в 1,5–2 раза, тогда как GOO_{comb} находит на порядок более быстрый план. Это показывает, что изменённый порядок перечисления DPhur *не устраняет* сильные ошибки стоимостной модели на JOB-Complex, а меняет их распределение.

Анализ причин.

1. *Теоретическое преимущество не реализуется на практике.* Свыше 90% времени t_{plan} приходится на процедуру построения путей, а не на само перечисление пар. DPhur и DPsize вызывают эту процедуру для одних и тех же допустимых пар, различаясь лишь порядком вызовов.

2. *Изменённый порядок перечисления ухудшает t_{exec} .* Это главная причина деградации. DPhur перечисляет пары обходом по соседям гиперграфа, а DPsize — в порядке возрастания размера подмножества. Как показано в секции «Архитектура планировщика PostgreSQL», при ошибках оценок кардинальностей стоимости конкурирующих путей попадают в ε -окрестность, и путь, поступивший в процедуру инкрементального отбора раньше, получает структурное преимущество. DPsize на каждом

уровне сначала обрабатывает левосторонние планы, где на каждом уровне присоединяются единичны отношения, для которых стоимостная модель PostgreSQL наиболее точна. DPhyp не имеет этого свойства: порядок перечисления определяется структурой гиперграфа и может отдавать приоритет ветвистым планам с менее надёжными оценками.

Вывод. Эксперимент с DPhyp показывает, что замена алгоритма перечисления при сохранении пространства поиска и процедуры построения путей не даёт систематического улучшения t_{total} . Причина — механизм ε -зависимости от порядка перебора: при ошибочных оценках кардинальностей стоимости конкурирующих путей попадают в ε -окрестность, и изменение порядка перечисления может как улучшить, так и ухудшить отдельные запросы, без устойчивого преимущества. Более того, если новый порядок систематически смещает приоритет в сторону планов с менее надёжными оценками, результат становится хуже DPsize.

Отсюда следуют два структурных ограничения для t_{total} , не устранимые заменой алгоритма перечисления:

- t_{exec} сильно зависит от качества оценок кардинальностей и порядком поступления путей в процедуру инкрементального отбора, а не алгоритмом обнаружения пар.
- t_{plan} определяется числом вызовов процедуры построения путей, которое при неизменном пространстве поиска совпадает у DPsize и DPhyp с точностью до фильтрации несвязных пар.

Для улучшения t_{total} необходимо либо сократить пространство поиска (фиксируя соединения с предсказуемыми оценками до основного DP), либо целенаправленно упорядочить перебор так, чтобы приоритет получали планы с надёжными оценками. Первое направление реализовано в подходе со

связыванием якорных участков (подход 5), второе — в модификации DPhyr, описанной далее (подход 4).

Следует подчеркнуть, что результат характеризует данную реализацию DPhyr, а не алгоритм в целом. Полноценная реализация с корректным построением обобщённых гиперрёбер и обнаружением конфликтов может показать иные результаты. Тем не менее пункты 1 (теоретическое преимущество не реализуется при $n \leq 17$) и 2 (зависимость от порядка перечисления) являются свойствами архитектуры PostgreSQL, а не конкретной реализации DPhyr.

4.5. Подход 4. Модифицированный DPhyr (DPhyrMod)

Мотивация. Эксперимент с DPhyr (подход 5) выявил три конкретные причины деградации: нарушение поуровневого инварианта PostgreSQL, невыгодный порядок обработки кандидатов и избыточные вызовы процедуры построения путей на промежуточных уровнях. Модифицированный DPhyr адресует каждую из этих причин отдельной модификацией, сохраняя оригинальный алгоритм перечисления `csg-cmp-пар` без изменений.

Модификация 1. Поуровневая материализация. Стандартный DPhyr строит отношения в порядке обхода гиперграфа, произвольно чередуя подпланы разных уровней. Однако архитектура планировщика PostgreSQL опирается на инвариант: все отношения уровня k должны быть полностью построены и финализированы до начала построения отношений уровня $k + 1$. Нарушение этого инварианта приводит к неполному распространению классов эквивалентности и пропуску путей.

Модификация разделяет работу DPhyr на две фазы:

1. **Перечисление.** Оригинальный алгоритм DPhyr перечисляет все csg-str-pairs, но не вызывает процедуру построения путей. Вместо этого для каждой пары (S_1, S_2) записывается список кандидатов в соответствующий узел DP-таблицы и вычисляется облегчённая оценка кардинальности $|\widehat{S_1 \bowtie S_2}|$.
2. **Материализация.** Узлы DP-таблицы группируются по уровню (числу базовых отношений) и обрабатываются строго последовательно: сначала все узлы уровня 2, затем уровня 3, и т. д. Для каждого узла вызывается процедура построения путей по всем его кандидатам, после чего выполняется финализация. Это восстанавливает поуровневый инвариант PostgreSQL.

Модификация 2. Приоритет левосторонние планов. Как показано в секции «Архитектура планировщика PostgreSQL», стандартный DPsize на каждом уровне обрабатывает левосторонние подпланы ($|S_1| = s - 1, |S_2| = 1$) до ветвистых. Это обеспечивает приоритет параметризованных путей (вложенный цикл с индексным сканированием) в процедуре инкрементального отбора. В DPhyr порядок кандидатов определяется обходом гиперграфа и не имеет этого свойства.

Модификация вводит два уровня сортировки:

1. **Между узлами одного уровня.** Узлы уровня k группируются по значению μ — минимальному размеру дочернего подплана среди всех кандидатов узла. Узлы с $\mu = 1$ (левосторонние) обрабатываются первыми, затем $\mu = 2$, и т. д.
2. **Внутри списка кандидатов одного узла.** Пары кандидатов сортируются по $\min(|S_1|, |S_2|)$ по возрастанию, так что левосторонние

планы узла поступают в процедуру построения путей раньше ветвистых. Обе сортировки воспроизводят порядок обработки стандартного DPsize в контексте DPhur-перечисления.

Модификация 3. Отсечение на промежуточных уровнях по кардинальности. Распределение числа узлов DP-таблицы по уровням имеет характерную форму нормального распределения с максимумом на средних уровнях ($s \approx n/2$). Для запроса с $n = 11$ таблицами на уровнях 5–8 сосредоточено $\sim 86\%$ всех узлов. Основная вычислительная нагрузка планирования приходится именно на эти уровни.

На каждом уровне из области горба ($\lceil n/3 \rceil + 1 \leq s \leq n - \lfloor n/3 \rfloor$) выполняется отсечение узлов с высокой оценкой кардинальности:

1. Вычисляется $\min_s |\widehat{S}|$ среди всех узлов уровня s .
2. Узлы с $|\widehat{S}| > \alpha \cdot \min_s |\widehat{S}|$ отсекаются (в реализации $\alpha = 300$).
3. Дополнительно отсекаются узлы с $|\widehat{S}| > C_{\text{out}}^{\text{GOO}}$, где $C_{\text{out}}^{\text{GOO}}$ — сумма кардинальностей промежуточных результатов жадного плана, вычисленная облегчённым GOO по оценкам $|\widehat{S}|$ из фазы перечисления. Если кардинальность одного промежуточного результата превышает суммарный объём всех промежуточных результатов жадного плана, такой подплан заведомо неоптимален.
4. **Гарантия покрытия.** После отсечения проверяется, что каждое базовое отношение $R_i \in \mathcal{R}$ входит хотя бы в один выживший узел уровня s . Если отношение потеряно, восстанавливается узел с минимальной $|\widehat{S}|$ среди отсечённых, содержащий это отношение. Это предотвращает отсечение всех путей через критически важные базовые отношения.

На крайних уровнях ($s \leq \lceil n/3 \rceil$ и $s > n - \lfloor n/3 \rfloor$) отсечение не выполняется:

число узлов мало, стоимостная модель ещё точна, а разнообразие путей критически важно для качества финального плана.

Результаты. Суммарные отношения к стандартному DPsize:

- t_{plan} : 0,966 (экономия 3,4%),
- t_{exec} : 0,967 (улучшение 3,3%),
- t_{total} : 0,967 (улучшение 3,3%).

Из 113 запросов 98 показали улучшение t_{total} , что значительно лучше, чем исходный DPhyp (78 из 113).

Абсолютные суммарные времена (по 113 запросам):

- DPsize: $t_{\text{plan}} = 52,3$ с, $t_{\text{exec}} = 88,8$ с, $t_{\text{total}} = 141,2$ с.
- DPhypMod: $t_{\text{plan}} = 50,5$ с, $t_{\text{exec}} = 85,9$ с, $t_{\text{total}} = 136,5$ с.
- DPhyp: $t_{\text{plan}} = 53,1$ с, $t_{\text{exec}} = 100,5$ с, $t_{\text{total}} = 153,6$ с.

Анализ: почему улучшение ограничено 3,3%.

1. *Пространство поиска совпадает с DPsize.* Все три модификации влияют на *порядок обработки* и *количество* узлов DP-таблицы, но не расширяют и не сужают множество рассматриваемых разбиений по сравнению с DPsize. Поуровневая материализация и сортировка кандидатов восстанавливают инварианты стандартного планировщика — но не превосходят их. DPhypMod *приближается* к DPsize по качеству планов, а не улучшает его.

2. *Отсечение экономит мало.* Запросы JOB содержат от 5 до 17 таблиц. При таких размерах число узлов DP-таблицы составляет от десятков до нескольких тысяч, а горб распределения — область, где отсечение действует — содержит от десятков до нескольких сотен узлов. Отсечение по кардинальности с консервативным порогом $\alpha = 300$ и гарантией покрытия

удаляет лишь единицы узлов на каждом уровне. Экономия t_{plan} от сокращения вызовов процедуры построения путей составляет 3,4%, что соответствует удалению $\sim 3\%$ узлов. Более агрессивное отсечение ($\alpha < 100$) рискует удалить узлы, через которые проходит оптимальный план, поскольку оценки кардинальностей на промежуточных уровнях ненадёжны.

3. Процедура построения путей остаётся узким местом. Свыше 90% времени t_{plan} приходится на процедуру построения путей, а не на перечисление или сортировку узлов. Замена алгоритма перечисления и оптимизация порядка обработки не могут существенно сократить число вызовов этой процедуры, потому что число допустимых разбиений определяется топологией графа соединений.

Для существенного улучшения необходимо сократить пространство поиска — уменьшая число отношений, подаваемых на вход DP, путём предварительного связывания подграфов с предсказуемыми свойствами. Этот вывод мотивирует переход к подходу со связыванием якорных участков.

Результаты на JOB-Complex. На JOB-Complex: $t_{\text{plan}} = 1,04$, $t_{\text{exec}} = 0,82$, $t_{\text{total}} = 0,83$, побед 10 из 30. Улучшение t_{exec} на 18% объясняется тем, что поуровневая материализация и сортировка кандидатов частично восстанавливают инварианты DPsize, но не устраняют катастрофические ошибки оценок на строковых соединениях. На запросах с умеренными ошибками (Q8, Q9, Q13, Q14) DPhypMod показывает результаты, сопоставимые с DPsize. На запросах с катастрофическими ошибками (Q5, Q11, Q12, Q20) он не достигает уровня GOO_{card} и GOO_{comb} , поскольку отсечение по $|\widehat{S}|$ на промежуточных уровнях неэффективно, когда сами оценки ошибочны на порядки.

4.6. Подход 5. Связывание якорных участков

Мотивация. Исходя из аннотации и выводов с прошлых подходов следует, что адаптивное планирование имеет смысл рассматривать как *предобработка графу* перед основным DP: сокращение числа вершин за счёт стягивания подграфов с надёжными оценками кардинальностей, после которой стандартный DPsize работает быстрее и устойчивее. Переключение между стратегиями происходит по единственному критерию $\#csg$, естественным образом отражающему обещанную в аннотации «сложность графа»: для простых графов применяется прямой DPsize, для сложных — связывание с последующим DPsize.

При этом связывание должно быть *безопасной*: стягиваться могут только те подграфы, для которых оценки кардинальностей заведомо надёжны, а центральные вершины не могут быть изолированы от своих соседей (это требование — прямое следствие провала подхода 2).

Определения. Пусть задан граф соединений $G = (\mathcal{R}, \mathcal{E})$ с n вершинами.

- **Якорная вершина** — вершина $v \in \mathcal{R}$, удовлетворяющая двум условиям:

1. $|v| < R_{\max}$
2. $|v| \cdot \alpha < \max_{u \in N(v)} |u|$

Якорь — это малая таблица, имеющая хотя бы одного соседа, который в α раз больше. Соединение якоря с крупным соседом высокоселективно, а его кардинальность оценивается надёжно, поскольку на единичных отношениях оценки более точные.

- **Якорный участок** — связная компонента подграфа, включающая в

себя якорную вершину и её соседей. Участок имеет ограниченную кардинальность:

1. $|v \cup N(v)| \leq L$, где v - якорная вершина

Если якорные участки смежны, т.е существует условие соединения между двумя вершинами из разных участков, то два участка объединяются в один.

Алгоритм. Входом является список базовых отношений и граф соединений. Алгоритм реализует каскад из трёх стратегий с убывающей точностью:

Шаг 1. Построение графа и оценка сложности. Строится граф соединений G . Для каждой связной компоненты C в G вычисляется $\#csg(C)$ — число связных подграфов, определяющее объём DP-таблицы и сложность перебора. Если $\#csg(C) \leq \tau$, граф передаётся стандартному DPsize без модификаций: при малом $\#csg$ точный перебор быстр.

Шаг 2. Связывание якорных участков. Если $\#csg(G) > \tau$, выполняется поиск и связание N итераций. Если на каком-то шаге условие перестает выполняться, то происходит досрочный выход из цикла итераций:

1. Для каждой вершины проверяется якорное условие. Все якоря и их соседи маркируются.
2. Удаление небезопасных вершин. Если вершина не является якорем, но входит в участок и содержит строковое соединение, то она удаляется из участка
3. Маркированные вершины разбиваются на связные компоненты обходом в ширину по подграфу маркированных вершин. Каждая связная компонента образует один якорный участок.

4. Каждый участок планируется *стандартным DPsize*.
5. Если кардинальность результата планирования участка превышает порог L , связывание участка отменяется: вершины возвращаются в граф как отдельные отношения.
6. Перестроение графа. Успешно спланированные участки заменяются новыми вершинами. Строится новый граф соединений и пересчитывается $\#csg$.

После связывания $\#csg$ пересчитывается. Если $\#csg \leq \tau$, новый граф передаётся DPsize.

Шаг 3. Жадный алгоритм. Если после связывания $\#csg > \tau$, оставшийся граф планируется комбинированным жадным алгоритмом GOO_{comb} .

Если граф имеет несколько компонент связности, они объединяются тем же GOO_{comb} в режиме декартова произведения.

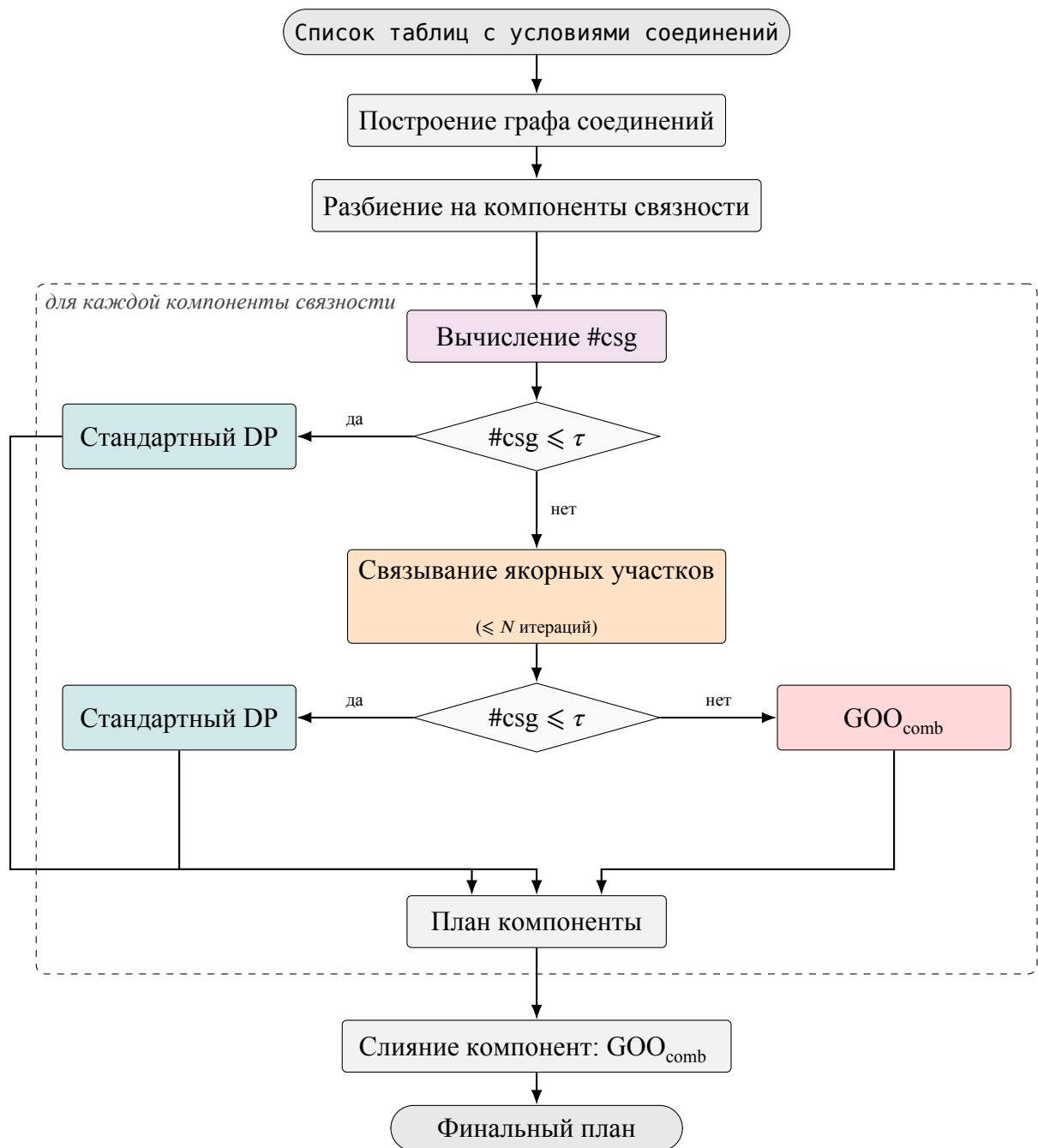


Рис. 4.1: Схема эвристического поиска порядка соединений: маршрутизация компонент связности по сложности графа #csg.

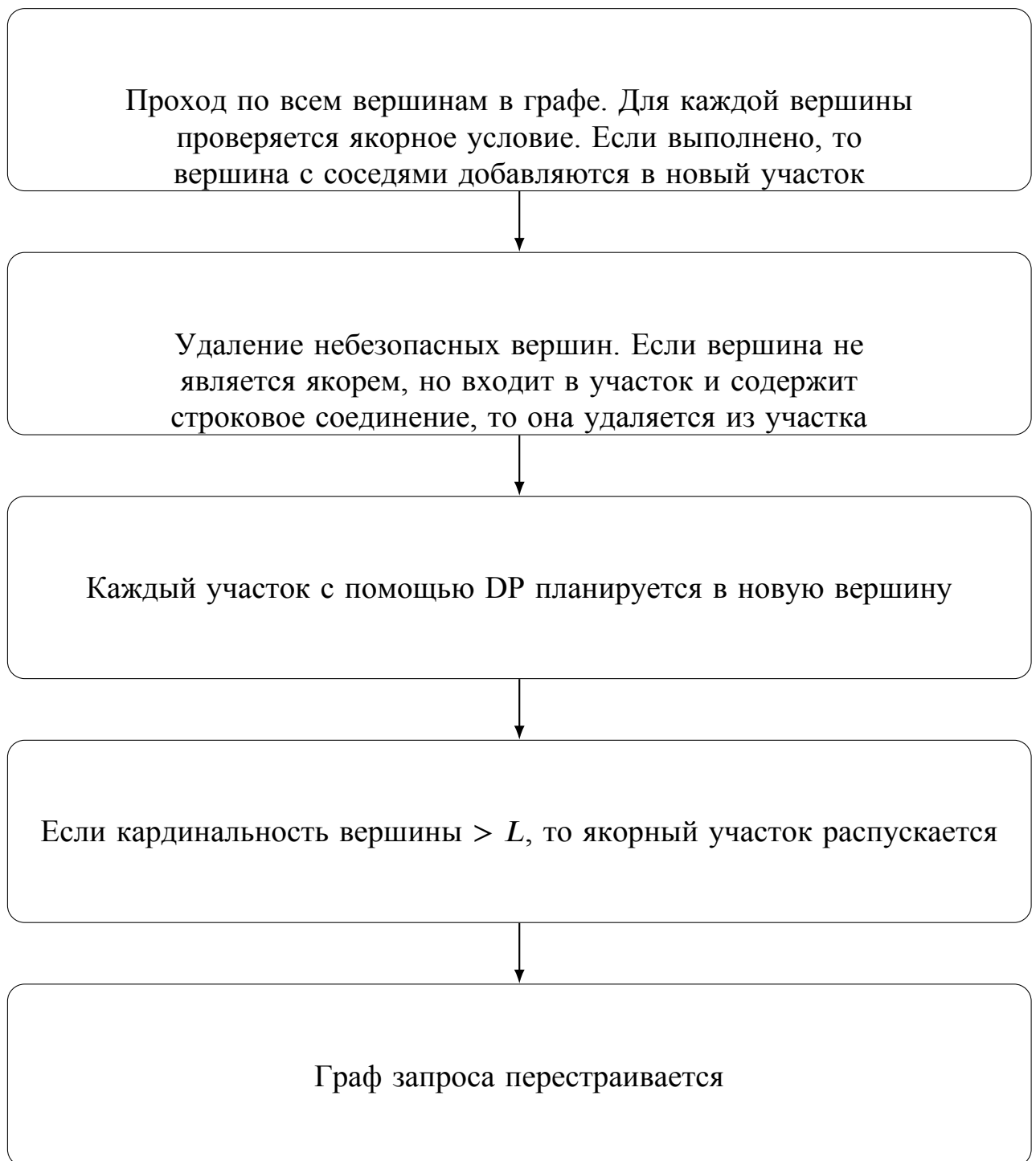


Рис. 4.2: Поиск и связывание якорных участков.

Выбор параметров. Подход использует четыре параметра: $\tau = 350$, $R_{\max} = 50\,000$, $\alpha = 100$, $L = 100\,000$.

Порог $\tau = 350$ определяет границу переключения между точным DP и якорным связыванием. См. приложение А.

Значение $R_{\max} = 50\,000$ отсекает все таблицы, сопоставимые по размеру с таблицами фактов для аналитических схем (10^5 – 10^7 строк). Множитель $\alpha = 100$ требует двухпорядкового различия между якорем и его крупнейшим соседом, ограничивая якоря справочниками и малыми таблицами, для которых соединения высокоселективны. Порог связывания $L = 100\,000$ обеспечивает, что новая вершина не вносит в граф отношение с большой кардинальностью. Стандартный DP внутри участка выбирает оптимальный план с высокой вероятностью, поскольку стоимостная модель надёжна для малых подпланов (см. секцию «Оценки в модели стоимости»). Проверка $|v \cup N(v)| \leq L$ является дополнительной защитой: если результат связания оказался неожиданно большим (признак ошибки оценки), связывание отменяется.

Все четыре значения подобраны экспериментально на JOB и не валидировались на других бенчмарках.

Защита центральных вершин. Свойство алгоритма: якорное условие $|v| < 50\,000 \wedge |v| \cdot 100 < \max_{u \in N(v)} |u|$ гарантирует, что центральные вершины (Например в IMD таблица title с $|t| = 2,5 \cdot 10^6$) *никогда* не становятся якорями. Центральная вершина может попасть в якорный участок только как сосед якоря — но в этом случае она входит в участок *вместе со всеми* соседями якоря, а не изолируется от части своих соседей. Это исключает ситуацию, приводящую к деградации у топологической декомпозицией.

Более того, в типичных запросах JOB центральная вершина имеет

степень, превышающую число вершин в якорном участке. Поскольку участок включает только соседей якоря, центральная вершина остаётся связанной с незатронутыми вершинами и сохраняет все параметризованные пути при последующем DPsize.

Результаты. Суммарные отношения к стандартному DPsize:

- t_{plan} : 0,778 (улучшение 22,2%),
- t_{exec} : 0,889 (улучшение 11,1%),
- t_{total} : 0,848 (улучшение 15,2%).

Из 113 запросов 105 показали улучшение t_{total} , 8 — незначительную деградацию. Время планирования улучшилось в 105 запросах, исполнения — в 101.

Абсолютные суммарные времена:

- DPsize: $t_{\text{plan}} = 52,3$ с, $t_{\text{exec}} = 88,8$ с, $t_{\text{total}} = 141,2$ с.
- Связывание якорных участков: $t_{\text{plan}} = 40,7$ с, $t_{\text{exec}} = 79,0$ с, $t_{\text{total}} = 119,7$ с.

Суммарная экономия — 21,5 секунды на 113 запросах, из которых 11,6 с приходится на t_{plan} и 9,9 с — на t_{exec} .

Анализ улучшения t_{plan} . Связывание уменьшает число вершин графа соединений, подаваемого на вход DPsize. Поскольку $\#csg(G)$ растёт экспоненциально с числом вершин, удаление даже 2–3 вершин может сократить $\#csg$ в несколько раз. Наибольшая экономия наблюдается на запросах с 13–17 таблицами:

- Запрос 33a (14 таблиц): t_{plan} уменьшилось с 627 мс до 98 мс (в 6,4 раза). Связывание сократила число вершин на 5–6, уменьшив $\#csg$ с нескольких тысяч до значения ниже порога $\tau = 350$.

- Запрос 29а (17 таблиц): t_{plan} уменьшилось с 2608 мс до 1292 мс (в 2,0 раза).
- Запрос 26а (12 таблиц): t_{plan} уменьшилось с 759 мс до 352 мс (в 2,2 раза).

Для запросов с ≤ 10 таблицами $\#csg$ уже невелик, и экономия t_{plan} составляет единицы процентов.

Анализ улучшения t_{exec} . Улучшение t_{exec} на 11,1% не является целью подхода, но является его следствием. Связывание фиксирует высокоселективные соединения (якорь + соседи) до основного DP. Внутри участка стандартный DP находит оптимальный план, поскольку участки малы и оценки кардинальностей для первых соединений надёжны. Новая вершина, созданная связыванием, несёт точную оценку кардинальности и фиксирует выбор физического пути (например, параметризованное индексное сканирование внутри участка). Это решение *не подвергается пересмотру* на последующих уровнях DPsize, где ошибки каскадируются и стоимостная модель менее надёжна. Фактически связывание изолирует наиболее предсказуемые решения от ϵ -шума на средних уровнях DP.

Анализ деградаций. Из 11 проигрышей 10 имеют деградацию менее 12%, а суммарные абсолютные потери составляют 1,8 секунды (против 23,3 секунды суммарного выигрыша). Наибольшая деградация — запрос 17а (t_{total} : 5098 $\rightarrow$ 6394 мс, +26,8%): t_{plan} увеличился с 278 до 354 мс, t_{exec} — с 4821 до 6040 мс. Причина: связывание выделило участок, которая в данном запросе не содержит наиболее селективного пути; новая вершина ограничивает DPsize в выборе альтернативного порядка, который оказался бы лучше.

Среди 13 запросов с деградацией t_{plan} причиной является накладной расход на построение графа соединений, вычисление $\#csg$ и планирование участков, который не окупается на запросах с $\#csg \leq \tau$, где стандартный

DPsize и так быстр.

Результаты на JOB-Complex. На 30 запросах JOB-Complex:

- t_{plan} : 0,85 (экономия 15%),
- t_{exec} : 0,59 (улучшение 41%),
- t_{total} : 0,61 (улучшение 39%).

Побед 20 из 30 (лучший показатель стабильности среди всех подходов).

Якорный подход корректно связывает участки на запросах 5, 11, 12, где DPsizes выбирает плохие планы с вложенными циклами, и находит хеш-соединения с t_{exec} на 2–3 порядка быстрее.

Однако GOO_{card} ($t_{\text{total}} = 0,11$) и GOO_{comb} (0,17) значительно превосходят якорный подход на JOB-Complex. Причина: якорный подход использует стандартный DPsizes для основной фазы планирования после связывания. На JOB-Complex стоимостная модель DPsizes является источником плохих решений на запросах со строковыми соединениями, и связывание не устраняет этот корень проблемы. В частности, запрос 19 демонстрирует деградацию t_{exec} в 86 раз: связывание создало вершину с ошибочной оценкой кардинальности для строкового соединения, которая приводит DP во вложенный цикл с $33 \cdot 10^6$ итераций.

Сравнение результатов JOB и JOB-Complex выявляет важное отличие. На JOB (умеренные ошибки кардинальностей) якорный подход — лучший среди всех протестированных. На JOB-Complex (катастрофические ошибки) жадные алгоритмы без стоимостной модели (GOO_{card} , GOO_{comb}) оказываются значительно лучше. t_{exec} сильно зависит от качества оценок кардинальностей. Когда оценки в несколько порядков ошибочны, любой подход, опирающийся на стоимостную модель, деградирует.

Результаты на истинных кардинальностях. В ходе исследования были также получены данные в условиях истинных кардинальностей. Для каждого запроса перебирались все связанные подграфы, каждому такому подграфу сопоставлялся подзапрос, для него вычислялась кардинальность. Таким образом каждому запросу из JOB и JOB-Complex, соответствует множество кардинальностей для всех его подзапросов. Было разработано расширение, которое подставляет истинные значения при оценке кардинальностей в планировщик PostgreSQL. Для эффективного доступа используется хэш таблица.

На наборе JOB суммарное время t_{plan} сократилось с 51,9 с до 40,5 с (−21,9%), t_{exec} — с 24,5 с до 22,1 с (−9,9%), а суммарное t_{total} — с 76,4 с до 62,6 с (−18,1%). Якорный подход оказался не хуже стандартного DPsize на 109 запросах из 113 (96,5%); деградации в оставшихся четырёх случаях не превышают 5,4%. На наборе JOB-Complex без запроса Q19 (29 запросов) улучшение составило −8,0% по t_{plan} , −47,3% по t_{exec} и −20,1% по t_{total} .

Главная компонента выигрыша — t_{plan} . Связывание заменяет подграфы с надёжными оценками одной вершиной, после чего DP работает на сокращённом графе с меньшим #csg. Безопасность выбора участка сохраняет качество плана, поэтому t_{exec} либо не меняется, либо снижается за счёт устранения локально субоптимальных порядков, которые DP выбирал из-за раздутого пространства поиска.

Иллюстративный пример — запрос Q29с из JOB, дающий наибольшую абсолютную экономию. На истинных кардинальностях DPsize строит план за 2,81 с при $t_{\text{exec}} = 17$ мс; якорный подход — за 0,91 с при $t_{\text{exec}} = 44$ мс. Граф запроса содержит около 18 вершин с несколькими малыми справочниками-якорями (role_type, info_type, comp_cast_type, keyword);

их соседи образуют четыре непересекающихся участка. После связания #csg нового графа падает ниже порога τ , и DPsize завершает перебор за время, сравнимое с накладными расходами на сам каскад. Выигрыш по t_{total} — 1,88 с (−66,4%).

Запрос Q19 из JOB-Complex демонстрирует обратный сценарий: t_{exec} возрастает в 1679 раз (120 мс vs 200,7 с) при ускорении t_{plan} на 31%. Стандартный DPsize выбирает план, в котором первым выполняется селективное хэш соединение с $\bowtie$ ak по предикату $s.name_rcode_nf = ak.name_rcode_nf$, сокращающий промежуточный результат до ~ 88 строк ещё до подключения крупных таблиц cast_info и name. В якорном варианте якорями стали справочники company_name, info_type, keyword; связывание охватило и ak, и s как соседей (оба связаны строковым предикатом с внутренними вершинами участка sn). Защита от строковых соединений проверяет внешние, а не внутренние рёбра участка и поэтому не сработала: рёбра $ak \leftrightarrow s$ и $ak \leftrightarrow sn$ оказались целиком внутри участка и были использованы локальным DPsize, который без знания о большой внешней таблице cast_info спланировал участок на промежуточный объём $\sim 19,5$ тыс. строк вместо ~ 88 . На внешнем уровне сжатый граф состоит всего из четырёх вершин (участок, ci, n, t). DPsize, видя участок, подключает к нему cast_info через вложенный цикл с ~ 34 млн повторных индексных сканирований.

5. Заключение

Для минимизации целевой функции в ходе исследования были реализованы в ядре PostgreSQL:

- Жадные подходы: GOO_{cost} , GOO_{card} , GOO_{comb} .
- Итеративная декомпозиция на топологии:
 1. Алгоритмы для обработки графового представления запроса.
 2. Варианты DP для разных топологий.
- Модификация алгоритма с открытым исходным кодом DPhyMod.
- Якорный подход.
- Расширение для планировщика PostgreSQL, которое с помощью высокоселективного доступа подставляет истинные значения на этапе оценки кардинальностей подпланов.

Экспериментальная оценка на двух бенчмарках позволяют сформулировать следующие результаты.

5.1. Сводные результаты

	JOB				JOB-Complex			
	t_{plan}	t_{exec}	t_{total}	побед	t_{plan}	t_{exec}	t_{total}	побед
Связывание якорных участков	0,78	0,89	0,85	105/113	0,85	0,59	0,61	20/30
Декомпозиция на топологии	0,83	3,55	2,55	32/113	0,82	3,12	2,99	12/30

5.2. Выводы

1. Замена алгоритма перебора подпланов, при сохранении функции построения и сравнения путей, не является достаточным средством минимизации t_{total} . DPhyp и его модификация DPhypMod рассматривают то же множество пар, что DPsize, и на JOB дают нейтральный или отрицательный результат. Это связано с тем, что нарушается инвариант, при котором левосторонние планы обрабатываются первыми. Обработка в первую очередь ветвистых планов может ухудшить t_{exec} , для ветвистых планов ошибки в оценках стоимости больше, чем в левосторонних.

Помимо этого, большая часть времени t_{plan} приходится на процедуру построения путей, число вызовов которой это $\#csg$. Для сокращения t_{plan} необходимо уменьшать само пространство поиска.

2. Качество оценок кардинальностей оказывает сильное влияние на любые алгоритмы DP. На JOB, где ошибки оценок умеренны, полный перебор DPsize даёт хорошие планы, и якорное связывание улучшает его. На JOB-Complex, где ошибки достигают 10^5 – 10^8 , DPsize выбирает планы в 11 раз медленнее оптимальных [11], и GOO_{card} — жадный алгоритм без стоимостной модели — оказывается в 9 раз лучше DPsize. Стоимостная модель PostgreSQL, построенная на предположении независимости атрибутов, при нарушении этого предположения направляет перебор в область плохих планов.

5.3. Направления дальнейшей работы

1. Динамический подбор параметров для якорного подхода. Подбор параметров во время планирования позволил бы сделать данный подход более гибким к разным классам запросов, схемам баз данных.

2. Обнаружение ненадёжных оценок. Разработка лёгковесной эвристики, маркирующей соединения с потенциально ненадёжными оценками (строковые столбцы, отсутствие первичного или внешнего ключа в соединении, высокий n_{distinct}), позволила бы использовать кардинальную метрику, а для надёжных — стоимостную, приближая реализацию к алгоритму $\mathcal{A}^*$, адаптивному не только по сложности графа, но и по качеству доступных статистик.

3. Использование стоимостных критериев и селективности для определения якорей и их участков на единичных отношениях.

4. Выработка других критериев для применения якорного подхода.

Список литературы

1. Ibaraki T., Kameda T. On the optimal nesting order for computing N-relational joins // ACM Trans. Database Syst. 1984. Т. 9, № 3. С. 482–502.
2. Leis V. и др. Query optimization through the looking glass, and what we found running the Join Order Benchmark // The VLDB Journal. 2018. Т. 27, № 5. С. 643–668.
3. Karp R.M. Reducibility among combinatorial problems // Complexity of computer computations. Plenum Press, 1972. С. 85–103.
4. Moerkotte G., Fender P., Eich M. On the correct and complete enumeration of the core search space // Proceedings of the 2013 ACM SIGMOD international conference on management of data. 2013.
5. Fegaras L. A new heuristic for optimizing large queries // Proceedings of the 9th International Conference on Database and Expert Systems Applications (DEXA). 1998.
6. Moerkotte G., Neumann T. Analysis of two existing and one new dynamic programming algorithm for the generation of optimal bushy join trees without cross products. 2006.
7. Moerkotte G., Neumann T. Analysis of two existing and one new dynamic programming algorithm for the generation of optimal bushy join trees without cross products // Proceedings of the 32nd International Conference on Very Large Data Bases (VLDB). 2006.
8. Moerkotte G., Neumann T. Dynamic programming strikes back // Proceedings of the 2008 ACM SIGMOD international conference on management of data. 2008.
9. Bennett K., Ferris M.C., Ioannidis Y.E. A genetic algorithm for database query

- optimization // Proceedings of the 4th International Conference on Genetic Algorithms. 1991.
10. Neumann T., Radke B. Adaptive optimization of very large join queries // Proceedings of the 2018 international conference on management of data (SIGMOD). 2018.
 11. Wehrstein J. и др. JOB-Complex: A challenging benchmark for traditional & learned query optimization // VLDB 2025 workshop: Applied AI for database systems and applications (AIDB). 2025.